Idea 20/50 - lottery_tickets_grokking


```
"Name": "lottery_tickets_grokking",
"Title": "Lottery Tickets in Grokking: Investigating Sparse Subnetworks
Capable of Algorithmic Learning",
"Experiment": "1. Implement an iterative magnitude pruning function for the
Transformer model. 2. Modify the training loop to support multiple rounds
of train-prune-reset cycles. 3. For each dataset, run experiments with
pruning levels of 30%
iteration, train the network to convergence, prune the specified percentage
of smallest weights, then reset remaining weights to their initial values.
5. Track metrics for each sparse network: time to grokking (or maximum
training time if grokking doesn't occur), final validation accuracy, and
training loss. 6. Introduce a 'grokking efficiency' metric: the ratio of
time to grokking for the sparse network vs. the dense network. For networks
that don't grok, use the maximum training time. 7. Plot learning curves for
each pruning level, highlighting grokking points and grokking efficiency.
8. Compare the structure of sparse networks that achieve grokking across
different operations, focusing on the distribution of preserved weights in
different layers and attention heads. 9. Analyze the correlation between
pruning level and grokking efficiency across different algorithmic tasks,
including cases where grokking fails to occur.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": false
```


Idea 21/50 - algebraic_structure_grokking


```
"Name": "algebraic_structure_grokking",
"Title": "Grokking and Algebraic Structure: How Group Properties Influence
Neural Network Learning",
"Experiment": "1. Implement new dataset classes for modular multiplication
and division (modulo p, where p is prime, ensuring proper group
structures). 2. For each operation (addition, multiplication, division),
calculate and store two properties: group order and number of generators.
3. Run experiments for each operation type, tracking: time to grokking,
final validation accuracy, and the two group properties. 4. Plot learning
curves and grokking points for each operation, labeled with their group
properties. 5. Analyze the correlation between group properties and
grokking behavior (e.g., time to grokking, steepness of accuracy
improvement). 6. Compare attention patterns across operations, focusing on
how they reflect the underlying group structure (e.g., uniformity for
commutative operations). 7. Test the model's ability to generalize by
evaluating on compositions of learned operations (e.g., a * b + c mod p)
after training on individual operations.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 22/50 - mdl_grokking


```
"Name": "mdl_grokking",
"Title": "Minimum Description Length and Grokking: Investigating the
Relationship Between Model Compression and Algorithmic Learning",
"Experiment": "1. Implement functions to calculate model complexity: (a) L2
norm of weights, (b) number of bits to store parameters at different
precisions, (c) effective number of parameters using BIC. 2. Modify the
training loop to track these complexity measures alongside existing
metrics. 3. Run experiments across all datasets, recording complexity
measures, validation accuracy, and training loss at regular intervals. 4.
Plot the evolution of model complexity alongside the grokking curve. 5.
Analyze the correlation between sudden decreases in model complexity and
the onset of grokking, including statistical tests for significance. 6.
Compare complexity dynamics across different operations and model sizes. 7.
Visualize weight distributions at pre-grokking, during grokking, and post-
grokking stages. 8. Implement and compare two early stopping mechanisms:
one based on model complexity stabilization and another based on validation
loss stabilization.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```


Idea 23/50 - invariance_learning_grokking


```
"Name": "invariance_learning_grokking",
"Title": "Learning Invariances in Grokking: Tracking Symmetry Awareness
During Algorithmic Learning",
"Experiment": "1. Modify AbstractDataset to generate transformed versions
of inputs (cyclic shifts for modular arithmetic, relabelings for
permutations). 2. Update the evaluation function to test model predictions
on both original and transformed inputs. 3. Implement an 'invariance score'
metric: mean absolute difference between predictions on original and
transformed inputs. 4. Modify the training loop to calculate and store the
invariance score at regular intervals. 5. Run experiments across all
datasets, tracking the invariance score alongside existing metrics. 6. Plot
the evolution of the invariance score alongside the grokking curve. 7.
Analyze how the invariance score changes before, during, and after
grokking. 8. Compare invariance learning across different operations and
model sizes. 9. Investigate correlation between invariance score and
generalization performance.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```


Idea 24/50 - grokking_double_descent


```
"Name": "grokking_double_descent",
"Title": "Grokking and Double Descent: Exploring the Intersection of Two
Deep Learning Phenomena",
"Experiment": "1. Create a range of model sizes by varying num_layers (1 to
8) and dim_model (32 to 512). 2. For each dataset, train models of
```

```
different sizes, tracking validation accuracy, training loss, and time to
grokking (99%
parameters to identify double descent behavior. 4. On the same plot, mark
the point where grokking occurs for each model size. 5. Analyze the
relationship between grokking timing and the different regimes of the
double descent curve (under-parameterized, critical, over-parameterized).
6. Calculate the correlation between model size and time to grokking. 7.
Compare double descent and grokking behavior across different operations
(modular arithmetic vs. permutations). 8. Investigate whether grokking
consistently occurs in a specific regime of the double descent curve.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": false
```

Idea 25/50 - ntk_alignment_grokking

```
"Name": "ntk_alignment_grokking",
"Title": "NTK-Output Alignment in Grokking: Tracking Feature Learning
Dynamics in Algorithmic Tasks",
"Experiment": "1. Implement a function to compute the NTK-output alignment:
the cosine similarity between the NTK's top eigenvector and the output
gradient. 2. Modify the training loop to compute and store this alignment
metric every 100 steps. 3. Run experiments across all datasets, tracking
NTK-output alignment alongside validation accuracy and training loss. 4.
Plot the evolution of NTK-output alignment alongside the grokking curve. 5.
Analyze how the alignment changes before, during, and after grokking,
identifying any consistent patterns across different operations. 6.
Investigate correlations between sudden changes in alignment and the onset
of grokking. 7. Compare alignment dynamics for models that achieve grokking
vs. those that don't. 8. Experiment with using the alignment metric as an
early stopping criterion or to adjust learning rates dynamically. 9.
Discuss implications of findings for understanding feature learning and
generalization in grokking.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 26/50 - loss_landscape_grokking

```
"Name": "loss_landscape_grokking",
"Title": "Loss Landscape Evolution in Grokking: Geometric Insights into
Algorithmic Learning",
"Experiment": "1. Implement functions to compute and visualize 2D loss
landscapes using filter-wise normalization. 2. Modify the training loop to
save model checkpoints at key points: start of training, 25%
just before grokking (based on validation accuracy), during grokking, and
after grokking. 3. For each checkpoint, compute and store 2D loss landscape
visualizations. 4. Define quantitative metrics for loss landscape
characteristics: (a) local smoothness (average gradient magnitude), (b)
global convexity (ratio of loss at edges to center), (c) barrier height
(maximum loss along minimum loss path). 5. Run experiments across all
datasets, generating loss landscapes and computing metrics at key points.
```